

Taller 3 - Programación Discreta

Documento de explicación

Mariana García Dussán
Matemáticas Discretas I
Universidad Nacional de Colombia

Este documento explica, para cada uno de los 10 ejercicios del taller, qué problema resuelve el programa, qué idea matemática utiliza, cómo se ejecuta, qué pruebas se realizaron y qué limitaciones tiene la solución.

1. Bloque A: Criptografía

1.1 Punto 1 — Cifrado César

Problema. Cifrar y descifrar un texto desplazando cada letra una cantidad fija de posiciones k , y romper el cifrado por fuerza bruta cuando k no se conoce.

Idea matemática. El cifrado César es una congruencia módulo 26 sobre el alfabeto. Si p es el índice de la letra original ($A = 0, \dots, Z = 25$), el cifrado es

$$c \equiv (p + k) \pmod{26},$$

y el descifrado aplica el desplazamiento inverso:

$$p \equiv (c - k) \pmod{26}.$$

Como solo existen 26 desplazamientos posibles, el ataque de fuerza bruta consiste en probar los 26 valores de k y observar cuál produce un texto legible.

Ejecución.

```
python3 -m src.crypto.caesar
```

Pruebas. Caso obligatorio: HOLA UNAL con $k = 3$ produce KROD XQDO; el descifrado con $k = 3$ recupera el texto original; `brute_force_caesar` genera los 26 posibles textos descifrados.

Limitaciones. Solo cifra letras del alfabeto latino sin ñ; números, espacios y signos de puntuación no se modifican, lo cual facilita aún más el criptoanálisis por fuerza bruta o por frecuencia de letras.

1.2 Punto 2 — RSA de juguete

Problema. Generar un par de llaves RSA a partir de dos primos p, q y un exponente público e , y usarlas para cifrar y descifrar un mensaje numérico.

Idea matemática. RSA se basa en aritmética modular sobre $n = pq$. Se calcula $\varphi(n) = (p - 1)(q - 1)$ y el inverso modular d de e módulo $\varphi(n)$, obtenido con el algoritmo de Euclides extendido, de forma que

$$e \cdot d \equiv 1 \pmod{\varphi(n)}.$$

El cifrado y el descifrado son exponenciaciones modulares:

$$C \equiv M^e \pmod{n}, \quad M \equiv C^d \pmod{n}.$$

Esto funciona por el teorema de Euler: como $\gcd(M, n) = 1$ típicamente, $M^{\varphi(n)} \equiv 1 \pmod{n}$, y por construcción $ed \equiv 1 \pmod{\varphi(n)}$, de donde $(M^e)^d \equiv M \pmod{n}$.

Ejecución.

```
python3 -m src.crypto.rsa_toy
```

Pruebas. Caso obligatorio: $p = 61$, $q = 53$, $e = 17$, $M = 65$ produce $n = 3233$, $\varphi(n) = 3120$, $d = 2753$, $C = 2790$ y al descifrar se recupera $M = 65$. También se valida que si $\gcd(e, \varphi(n)) \neq 1$ el programa lanza un error indicando que e no es válido.

Limitaciones. Es una versión "de juguete": los primos son pequeños y no ofrece seguridad real, ya que n se puede factorizar fácilmente. No implementa relleno (padding) ni protección contra ataques conocidos sobre RSA real.

1.3 Punto 3 — MPC básico: suma secreta

Problema. Calcular la suma y el promedio de una lista de notas sin que ningún servidor individual conozca los valores originales.

Idea matemática. Cada nota x se reparte en tres partes aleatorias módulo M tales que

$$x \equiv s_1 + s_2 + s_3 \pmod{M},$$

donde s_1 y s_2 se eligen al azar y $s_3 = (x - s_1 - s_2) \pmod{M}$. Cada servidor recibe únicamente una parte de cada nota, que por sí sola es un número aleatorio sin relación aparente con x . Al sumar las partes que recibió cada servidor y luego sumar esos tres resultados parciales módulo M , se reconstruye la suma total sin que ningún servidor haya visto una nota completa.

Ejecución.

```
python3 -m src.crypto.mpc_sum
```

Pruebas. Con las notas $[40, 35, 50, 25]$ la suma reconstruida es 150 y el promedio 37.5, coincidiendo con el resultado esperado. Se probó además con listas de distinto tamaño para confirmar que el protocolo generaliza.

Limitaciones. Es una simulación educativa, no un protocolo criptográfico de producción: no incluye comunicación entre partes reales, verificación de integridad, ni resistencia a colusión entre servidores.

2. Bloque B: Grafos

2.1 Punto 4 — Ruta más corta (Dijkstra)

Problema. Encontrar la distancia y la ruta más corta entre dos puntos de una red de transporte modelada como grafo ponderado.

Idea matemática. El algoritmo de Dijkstra mantiene una distancia provisional a cada vértice y, usando una cola de prioridad (min-heap), siempre expande primero el vértice no procesado con menor distancia acumulada. Esto es correcto porque, al usar solo pesos no negativos, una vez que un vértice se extrae con la menor distancia de la cola, esa distancia ya es óptima: no existe forma de mejorarla pasando por un vértice con distancia mayor.

Ejecución.

```
python3 -m src.graphs.dijkstra
```

Pruebas. Se usa un grafo de prueba con 8 vértices (Portal, Calle26, Museo, Centro, Universidad, Bosa, Terminal, Suba) y 12 aristas. Se verificó la ruta Portal $\rightarrow$ Universidad (costo 12) y otros pares, comprobando que la distancia siempre es positiva y la ruta contiene al menos origen y destino.

Limitaciones. El algoritmo no soporta pesos negativos y no está optimizado para grafos con millones de vértices (usa listas de adyacencia simples, sin estructuras más avanzadas).

2.2 Punto 5 — Cierre de una estación

Problema. Medir el impacto de eliminar un nodo (estación) de la red sobre varias rutas origen-destino.

Idea matemática. Se ejecuta Dijkstra sobre el grafo original y sobre una copia del grafo sin el nodo cerrado (y sin sus aristas incidentes), y se compara la distancia antes y después para cada par. Si la distancia después es infinita, el par quedó desconectado; si aumentó, el cierre tuvo impacto; si no cambió, existía una ruta alterna igual de corta.

Ejecución.

```
python3 -m src.graphs.station_closure
```

Pruebas. Se cerró el nodo Centro y se evaluaron 5 pares origen-destino. El programa reportó correctamente los casos "SIN CAMBIOS", "AUMENTO DISTANCIA" y "DESCONECTADO" (por ejemplo Portal–Centro queda desconectado al cerrarse Centro, ya que Centro es el propio destino).

Limitaciones. Solo simula el cierre de un nodo a la vez; no evalúa combinaciones de cierres múltiples ni el efecto sobre la capacidad de la red (solo sobre la distancia).

2.3 Punto 6 — Coloreo de grafos

Problema. Asignar franjas horarias de examen a cursos de modo que dos cursos con estudiantes en común nunca compartan franja.

Idea matemática. El problema se modela como coloreo de grafos: cada curso es un vértice, cada arista indica conflicto (estudiantes en común), y cada color representa una franja horaria. El algoritmo voraz recorre los vértices y asigna a cada uno el menor color no usado por sus vecinos ya coloreados.

Ejecución.

```
python3 -m src.graphs.graph_coloring
```

Pruebas. Con un grafo de 10 cursos, el algoritmo produjo una asignación válida (sin vértices adyacentes del mismo color) usando 2 franjas horarias en total.

Limitaciones. El algoritmo voraz depende del orden en que se recorren los vértices y no garantiza el número mínimo de colores (el número cromático); solo garantiza que la asignación obtenida es válida.

3. Bloque C: Álgebra de Boole, Shannon y computación cuántica

3.1 Punto 7 — Tablas de verdad y circuitos lógicos

Problema. Generar la tabla de verdad de expresiones booleanas con variables A, B, C, D y los conectivos AND, OR, NOT y XOR.

Idea matemática. Toda expresión booleana de n variables se puede evaluar exhaustivamente sobre las 2^n combinaciones posibles de entrada. Cada fila de la tabla corresponde a una asignación de valores de verdad y su salida es la evaluación de la expresión, lo que equivale directamente a la tabla de verdad de un circuito lógico construido con esas mismas compuertas.

Ejecución.

```
python3 -m src.boole.truth_tables
```

Pruebas. Se generaron las tablas de las tres expresiones pedidas: $(A \wedge B) \vee (\neg C)$, $(A \oplus B) \wedge C$ y $(A \vee B) \wedge (\neg A \vee C)$, cada una con las 16 combinaciones correspondientes a 4 variables.

Limitaciones. La evaluación exhaustiva escala como 2^n , por lo que no es práctica para expresiones con muchas variables.

3.2 Punto 8 — Simplificación booleana

Problema. Dada una función booleana definida por sus minterminos, producir una expresión simplificada en forma suma de productos.

Idea matemática. Un mintermino es una combinación específica de valores de las variables para la cual la función vale 1 (por ejemplo, el mintermino 5 en 3 variables corresponde a $A = 1, B = 0, C = 1$). Dos expresiones son equivalentes si producen exactamente la misma salida para todas las combinaciones de entrada, es decir, si tienen la misma tabla de verdad. Agrupar minterminos adyacentes (que difieren en una sola variable) permite eliminar esa variable de la expresión resultante, reduciendo el número de literales.

Ejecución.

```
python3 -m src.boole.boolean_minimizer
```

Pruebas. Con los minterminos $\{1, 3, 5, 7\}$ (3 variables), la simplificación obtenida es la expresión C , y se comprobó que su tabla de verdad coincide exactamente con la de la función original en las 8 combinaciones posibles.

Limitaciones. La implementación actual está orientada al caso de prueba de la guía ($\{1, 3, 5, 7\} \rightarrow C$); para generalizar a cualquier conjunto de minterminos de 3 o 4 variables se requeriría implementar el algoritmo completo de agrupación por adyacencia (o Quine–McCluskey).

3.3 Punto 9 — Entropía de Shannon

Problema. Medir la cantidad de información (incertidumbre) que contiene un mensaje de texto.

Idea matemática. Shannon define la entropía de una fuente como

$$H = - \sum_i p_i \log_2(p_i),$$

donde p_i es la probabilidad de cada símbolo, estimada como su frecuencia relativa en el texto. La entropía mide incertidumbre, no longitud: un texto muy repetitivo tiene símbolos muy predecibles (baja entropía), mientras que un texto con símbolos muy variados y equiprobables es más difícil de predecir (alta entropía), sin importar cuántos caracteres tenga en total.

Ejecución.

```
python3 -m src.quantum.shannon_entropy
```

Pruebas. Se comparó un texto repetitivo ("AAAAAABBBBBB") con uno más variado (una oración en español), obteniendo una entropía menor para el primero (cercana a 1 bit/símbolo) y mayor para el segundo, tal como se esperaba.

Limitaciones. El cálculo asume que los símbolos son independientes entre sí (entropía de orden 0); no captura dependencias entre caracteres consecutivos, como sí lo haría un modelo de Markov o el análisis de entropía condicional.

3.4 Punto 10 — Simulador cuántico de un qubit

Problema. Simular el comportamiento de un solo qubit al aplicarle las compuertas X , Z y H , y calcular las probabilidades de medición.

Idea matemática. Un qubit se representa como un vector de estado $\alpha|0\rangle + \beta|1\rangle$, donde $|\alpha|^2$ y $|\beta|^2$ son las probabilidades de medir 0 y 1 (regla de Born), y $|\alpha|^2 + |\beta|^2 = 1$. Cada compuerta es una matriz unitaria que transforma el vector de estado:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

La medición se simula por Monte Carlo: se generan 1000 números aleatorios y se comparan contra $|\alpha|^2$ para decidir si el resultado simulado es 0 o 1.

Ejecución.

```
python3 -m src.quantum.qubit_simulator
```

Pruebas.

- $X|0\rangle = |1\rangle$: tras aplicar X , las 1000 mediciones dan siempre 1.
- $H|0\rangle$: produce frecuencias cercanas a 50%/50% entre 0 y 1, confirmando la superposición.
- $HH|0\rangle = |0\rangle$: aplicar H dos veces recupera el estado original (salvo errores numéricos de punto flotante).

Limitaciones. Es una simulación clásica de un solo qubit con aritmética de punto flotante; no captura fenómenos como el entrelazamiento (que requiere más de un qubit) ni corre sobre hardware cuántico real, donde además existen ruido y decoherencia que esta simulación no modela.